

googLeadsagent.ai

The Complete 5-Day AI Agents Crash Course

From Absolute Zero to Deploying Multi-Agent Systems

A Step-by-Step Tutorial for People Who've Never Written a Line of Code

John Williams

Senior Paid Media Specialist • 15+ Years • \$48M+ in Ad Spend Managed
Seer Interactive • Creator of googleadsagent.ai • It All Started With A Idea

28 API Actions • 6 Sub-Agents • Built on Claude by Anthropic
Google/Kaggle White Papers • MCP + A2A Protocols • 5 Vibe Coding Game Projects

Version 3.0 • February 2026 • Free & Open Source

Table of Contents

Preface

- Why This Course Exists
- Intended Audience and Learning Paths
- The googleadsagent.ai Through-Line
- Curriculum Overview
- APA Citation Note

Section 0: Complete Environment Setup

- Part A: Claude AI (Primary Assistant)
 - Interface Orientation
 - Pricing Comparison
 - Verification Test
- Part B: ChatGPT (Secondary Assistant)
- Part C: Cursor IDE (Development Environment)
 - Essential Cursor Features
- Part D: Kaggle (White Papers & Notebooks)
- Part E: GitHub
- Part F: Python 3.12+ (Builder & Architect Paths)
- Pre-Course Verification Checklist

Day 1: Introduction to AI Agents

Learning Objectives

Upon completing this day, learners will be able to:

- Define what an AI agent is and distinguish it from a chatbot
- Identify the three core components of agent architecture (Brain, Tools, Loop)
- Explain the ReAct loop and its role in autonomous reasoning
- Compare five agent architecture patterns and their optimal use cases
- Build a functional game using vibe coding (natural-language prompting)

Defining AI Agents

The Three Components of Agent Architecture

- Component 1: The Brain (Foundation Model)

- Component 2: The Tools

- Component 3: The Loop (Orchestration / ReAct)

Chatbot vs. Agent: Definitive Comparison

Agent Architecture Patterns

Required Reading

Day 1 Resource Matrix

Hands-On Lab 1A: Agent-Style Reasoning

Vibe Coding Project: Tetris

- Build in Claude (Recommended)

- Alternative: Build in Cursor

Key Takeaways

- AI agents differ from chatbots in autonomy, tool access, and iterative reasoning.
- Every agent has a Brain (LLM), Tools (executable functions), and a Loop (ReAct orchestration).
- googleadsagent.ai demonstrates all three: Claude as Brain, 28 API actions as Tools, The Loop as orchestration.
- Vibe coding is the foundational skill for modern agent development.

Day 1 Self-Assessment

Day 2: Agent Tools & Interoperability With MCP

Learning Objectives

Upon completing this day, learners will be able to:

- Explain how agents use tools to interact with external systems
- Trace the six-step tool-use execution cycle
- Define MCP and its three architectural components
- Analyze the strategic implications of universal tool interoperability
- Design a tool schema suitable for an MCP server

Understanding Agent Tools

The Tool-Use Execution Cycle

Tool Taxonomy in googleadsagent.ai

Model Context Protocol (MCP)

MCP Architecture

Day 2 Resource Matrix

Hands-On Lab 2A: Observing Tool Use

Hands-On Lab 2B: Designing a Tool Schema

Vibe Coding Project: Zelda-Style Adventure

Key Takeaways

- Tools transform an LLM from a text generator into an agent that can act on the world.
- MCP standardizes how AI connects to tools, eliminating per-application custom code.
- The tool-use cycle: goal, reasoning, structured call, execution, result injection, synthesis.
- MCP adoption by Google, OpenAI, and Microsoft signals convergence on a universal standard.

Day 2 Self-Assessment

Day 3: Context Engineering — Sessions & Memory

Learning Objectives

Upon completing this day, learners will be able to:

- Define context engineering and distinguish it from prompt engineering
- Identify the six layers of agent context with concrete examples

- Explain the scratchpad pattern for multi-step workflows
- Describe RAG and its role in grounding agent responses
- Compare five memory management strategies and their tradeoffs

Defining Context Engineering

The Six Layers of Agent Context

The Scratchpad Pattern

RAG: Retrieval-Augmented Generation

Memory Management Strategies

Day 3 Resource Matrix

Vibe Coding Project: Football (Tecmo Bowl)

Key Takeaways

- Context engineering is complete information management, far broader than prompt engineering.
- Agents operate across six context layers: system, user, short-term, long-term, RAG, tool results.
- The scratchpad pattern provides optimal efficiency-accuracy balance for multi-step workflows.
- googleadsagent.ai's CEP Protocol exemplifies context engineering in practice.

Day 3 Self-Assessment

Day 4: Agent Quality — Evaluation & Observability

Learning Objectives

Upon completing this day, learners will be able to:

- Name the three pillars of agent quality
- Explain the LLM-as-a-Judge evaluation methodology
- Describe input, output, and action guardrails with examples
- Analyze why hallucination is uniquely dangerous in advertising
- Identify key observability metrics for production monitoring

The Agent Quality Problem

Three Pillars of Agent Quality

Pillar 1: Evaluation

Pillar 2: Observability

Pillar 3: Guardrails

LLM-as-a-Judge Evaluation

Day 4 Resource Matrix

Vibe Coding Project: Super Mario Bros

Key Takeaways

- Quality requires three pillars: Evaluation (testing), Observability (monitoring), Guardrails (safety).

- LLM-as-a-Judge enables scalable evaluation of agent outputs against defined rubrics.
- Hallucination in advertising agents causes direct financial loss, making quality non-negotiable.
- Write-safety protocols like CONFIRM create human checkpoints before data mutations.

Day 4 Self-Assessment

Day 5: Prototype to Production & Multi-Agent Systems

Learning Objectives

Upon completing this day, learners will be able to:

- Contrast prototype and production deployment requirements
- Define A2A and explain how it complements MCP
- Trace a multi-agent workflow through googleadsagent.ai's 6 sub-agents
- Explain Google's Universal Commerce Protocol and its implications
- Architect a complete multi-agent system

The Prototype-to-Production Gap

Agent-to-Agent Protocol (A2A)

MCP vs. A2A: Complementary Protocols

googleadsagent.ai: Architecture Walkthrough

Google's Agentic Commerce Vision

Day 5 Resource Matrix

Vibe Coding Project: Duck Hunt

Capstone Project: AI-Powered Game Master RPG

Key Takeaways

- Production requires error handling, auth, monitoring, scaling, testing, privacy, cost management, and human oversight.
- MCP connects agents to tools; A2A connects agents to each other. They are complementary.
- Multi-agent systems use specialized sub-agents coordinated by an orchestrator.
- Google's UCP signals a future where AI agents participate directly in commerce.

Day 5 Self-Assessment

Appendix A: Complete Resource Library

Google/Kaggle White Papers (Free)

Free Courses and Certifications

Protocols and Specifications

Agent Development Frameworks

Observability and Evaluation Platforms

Vibe Coding Game Projects

Official Citations Index

Appendix B: Glossary of Key Terms

Appendix C: googleadsagent.ai Technical Reference

System Overview
Sub-Agent Specifications
Complete Action Directory (28 Actions)

Preface

Why This Course Exists

In 15+ years of managing over \$48 million in digital advertising spend across Google Ads, Meta, Microsoft, and Amazon—at companies including NortonLifeLock, Gen Digital, Avast, and Farmers Insurance—I have watched the paid media industry transform three times. The rise of programmatic buying. The shift to machine-learning bidding. And now, the agentic era.

This third shift is the largest. The industry is moving from artificial intelligence that suggests to artificial intelligence that acts—from tools that surface data to agents that analyze, decide, and execute. Google CEO Sundar Pichai declared at Google I/O 2025 that agentic capabilities represent “the direction where we are investing the most,” and during the Q2 2025 earnings call he predicted that 2026 would be “the year in which people kind of use agentic experiences more broadly” (Alphabet Inc., 2025).

I built googleadsagent.ai—an open-source, enterprise-grade AI agent with 28 custom Python API actions and 6 specialized sub-agents (Simba, Nemo, Elsa, Aladdin, Moana, and Baymax) that manages Google Ads accounts through live API access, built on Claude by Anthropic—because I recognized this future was arriving faster than most practitioners realized.

This course is free, open source, and designed for professionals at every level—from individuals who have never opened a code editor to experienced developers seeking a structured entry point into agent architecture. Every concept is explained with accessible analogies. Every technical step includes a “What You Should See” verification callout. Every day concludes with a hands-on project, including fully playable video games constructed through natural-language prompting.

CITATION

“We think of agents as systems that combine the intelligence of advanced AI models with access to tools, so they can take actions on your behalf and under your control.” — Google I/O 2025 Keynote

— Sundar Pichai, CEO of Google and Alphabet <https://blog.google/innovation-and-ai/technology/ai/io-2025-keynote/>

CITATION

“Over the last year, we have been investing in developing more agentic models, meaning they can understand more about the world around you, think multiple steps ahead, and take action on your behalf.”

— Gemini 2.0 Launch

— Demis Hassabis, CEO of Google DeepMind <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/>

Intended Audience and Learning Paths

This course serves three distinct audiences. Readers should select the path that matches their experience level and goals. All paths cover identical conceptual material; they differ in the depth of hands-on implementation.

Path	Profile	Focus	Time per Day	Prerequisites
 Observer	Executives, strategists, curious learners	Concepts and implications only	2–3 hours	None
 Builder	Marketers, analysts, career changers	Hands-on labs and game projects	4–6 hours	Basic computer literacy
 Architect	Developers, engineers, technical leads	Deep theory, code, and production deployment	6–8 hours	Programming familiarity

The googleadsagent.ai Through-Line

Throughout this course, googleadsagent.ai serves as the primary reference implementation. Every abstract concept maps to a concrete component of the system:

Course Concept	googleadsagent.ai Implementation	Course Day
The Brain (LLM)	Claude by Anthropic (Opus 4.5 and Sonnet 4.5 models)	Day 1
Tools	28 custom Python API actions with live Google Ads API read/write access	Day 2
MCP Interoperability	Tool executor adapter layer (tool_executor.py)	Day 2
Context / Memory	CEP Protocol + session state manager (Action #17)	Day 3
Quality / Validation	Top-down reporting with account-total cross-validation	Day 4
Multi-Agent Orchestration	6 Disney-named sub-agents:  Simba,  Nemo,  Elsa,  Aladdin,  Moana,  Baymax	Day 5
Write Safety	CONFIRM protocol with current-vs-proposed preview + rollback labeling	Day 4
Production Architecture	CLI, REST API, Docker, Python SDK deployment options	Day 5

Curriculum Overview

Day	Topic	Google/Kaggle White Paper	Game Project
Setup	Complete A-Z Environment Configuration	—	—

Day 1	Introduction to AI Agents	Introduction to Agents	 Tetris
Day 2	Agent Tools & Interoperability with MCP	Agent Tools & MCP	 Zelda-Style Adventure
Day 3	Context Engineering: Sessions & Memory	Context Engineering	 Football (Tecmo Bowl)
Day 4	Agent Quality: Evaluation & Observability	Agent Quality	 Super Mario Bros
Day 5	Prototype to Production & Multi-Agent Systems	Prototype to Production	 Duck Hunt + Capstone RPG

How to Use This Course

This course is structured as a five-day intensive. Each day builds on previous concepts, but also functions as standalone reference.

Sequential learner: Complete Section 0, then Day 1 through Day 5. Recommended for first-time learners.

Reference user: Use the Table of Contents to navigate directly. Each section is self-contained with its own resource matrix.

Workshop facilitator: Each day includes labs, game projects, and self-assessment questions for classroom delivery.

Conference attendee: The Executive Summary provides a concise overview. Appendix B (Glossary) and Appendix A (Resources) are quick-reference.

Acknowledgments

This course draws on foundational work from researchers and organizations advancing AI agents. Recognition is due to Google's Kaggle team (Julia Wiesinger, Patrick Marlow, Vladimir Vuskovic) for the white paper series forming this curriculum's backbone. Anthropic's MCP and Claude work, Google's A2A team, and the open-source communities (LangChain, CrewAI, AutoGen) made this field accessible. Microsoft's AI Agents for Beginners provided pedagogical reference. Colleagues at Seer Interactive and Casteel High School coaching staff sharpen my thinking on strategy and execution.

APA Citation Note

This document follows APA 7th Edition formatting conventions. All external sources are cited in-text with author-date format and linked where available. Direct quotations from keynote addresses, white papers, and official documentation are attributed with full source information.

Section 0: Complete Environment Setup

Before beginning Day 1, all required tools must be installed and verified. This section provides step-by-step instructions for each platform, including verification checks at every stage. Estimated time: 30–45 minutes.

Part A: Claude AI (Primary Assistant)

Claude, developed by Anthropic, serves as the primary AI assistant throughout this course and is the foundation model powering googleadsagent.ai.

1. Navigate to claude.ai in any modern browser (Chrome, Safari, Firefox, or Edge).
2. Select “Sign Up.” Choose “Continue with Google” for the fastest setup, or “Continue with email” for non-Gmail accounts.
3. If using email registration: enter your address, retrieve the 6-digit verification code from your inbox, and enter it on the Claude page.
4. Complete phone verification by entering your mobile number and the SMS code you receive. One phone number per account; users must be 18 or older.
5. Accept the Terms of Service and click Continue.
6. The Claude chat interface loads. You are ready.

WHAT YOU SHOULD SEE

A clean interface with a text input field reading “What can I help you with?” at the bottom center. Left sidebar displays “New Chat” button and conversation history. Main area is empty. Your account icon appears in the bottom-left corner.

Interface Orientation

Chat Input (bottom center): Type messages (“prompts”) and press Enter. This is where all interaction occurs.

Sidebar (left): Conversation history. Click “+” or “New Chat” to begin a fresh thread.

File Uploads (paperclip icon): Attach images, PDFs, spreadsheets, or code files for Claude to analyze. Up to 5 files per message.

Artifacts (right panel): When Claude generates code, documents, or interactive applications, a live preview renders on the right. This is where you will play games you build.

Projects: Organize related conversations with persistent reference files.

Pricing Comparison

Feature	Free Tier	Pro (\$20/month)
Model Access	Claude Sonnet (highly capable)	Claude Opus (most intelligent)
Daily Messages	Approximately 30–45	5× increased usage
File Uploads	Included	Included
Artifacts (live previews)	Included	Included

Projects	Included	Included
Web Search	Included	Included

**PRO TIP**

The free tier is sufficient for this entire course. Early prototypes of googleadsagent.ai were developed using the free tier.

Verification Test

Copy and paste the following prompt into Claude to confirm your setup:

```
You are a Research Agent. Explain what an AI agent is in
3 sentences as if speaking to a colleague at a coffee shop,
then provide one real-world example of an AI agent in action.
```

**WHAT YOU SHOULD SEE**

Claude responds with a clear, conversational explanation of AI agents and provides a concrete example. Your prompt appears above the response. If you see this, your Claude setup is complete.

Part B: ChatGPT (Secondary Assistant)

1. Navigate to chatgpt.com.
2. Select “Sign up” and choose Google, Apple, Microsoft, or email authentication.
3. Complete email and phone verification as prompted.
4. The ChatGPT interface loads with a centered message input.

**WHAT YOU SHOULD SEE**

A chat interface with “Message ChatGPT” input field. Model selector dropdown (GPT-4o) appears at the top. Left sidebar shows conversation history.

Part C: Cursor IDE (Development Environment)

Cursor is a code editor with integrated AI capabilities. Users describe desired functionality in natural language, and the AI generates the code—a methodology termed “vibe coding” by Andrej Karpathy (Karpathy, 2025).

**CITATION**

“It’s not really coding—I just see stuff, say stuff, run stuff, and copy-paste stuff, and it mostly works.”
— Andrej Karpathy, Former Director of AI at Tesla <https://x.com/karpathy>

1. Navigate to cursor.com and select “Download.” The site auto-detects your operating system.
2. Install: macOS—drag to Applications. Windows—run the .exe installer. Linux—execute the ApImage.
3. Launch Cursor and sign in via the gear icon (bottom-left) using Google or email.
4. Select your preferred theme (Dark or Light).

**WHAT YOU SHOULD SEE**

A three-panel editor: Left = File Explorer (project file tree). Center = code editor (empty or welcome tab). Bottom = Terminal. Press Cmd+I (macOS) or Ctrl+I (Windows) to open the AI Chat panel on the right.

Essential Cursor Features

AI Chat Panel (Cmd+I / Ctrl+I): Natural-language interface for code generation. Describe what you want; the AI writes it.

Agent Mode (default): Full read/write access to project files, terminal commands, and multi-file editing.

File Explorer (left): Displays project structure. Right-click HTML files and select “Reveal in Finder/Explorer” to open in browser.



PRO TIP

The googleadsagent.ai front-end interface was built primarily through Cursor’s AI Chat panel, with the AI generating UI components, API connections, and data visualizations while the developer focused on architectural decisions.

Part D: Kaggle (White Papers & Notebooks)

1. Navigate to kaggle.com and select Register.
2. Sign in with Google or create an email-based account.
3. **Verify your phone number** via Settings → Phone Verification. This step is required to execute interactive code notebooks.



CRITICAL

Phone verification on Kaggle is mandatory for running the interactive Codelabs that accompany the Google/Kaggle white papers. Do not skip this step.

Part E: GitHub

1. Navigate to github.com and select Sign Up.
2. Create an account with email, password, and username. Verify email.
3. Bookmark the following repositories:
 - Google White Papers Collection: github.com/sameeerjadhav/google-agents-resources
 - Microsoft AI Agents for Beginners: github.com/microsoft/ai-agents-for-beginners
 - A2A Protocol: github.com/a2aproject/A2A
 - MCP SDKs: github.com/modelcontextprotocol

Part F: Python 3.12+ (Builder & Architect Paths)

1. Navigate to python.org/downloads and download Python 3.12 or later.
2. Check “Add Python to PATH” before clicking Install Now. Omitting this checkbox will prevent all subsequent Python operations.
3. Verify installation: open Terminal (macOS) or Command Prompt (Windows) and type: `python --version`



WHAT YOU SHOULD SEE

Terminal displays: Python 3.12.x (where x is any patch number). If an error appears, reinstall Python with the PATH checkbox selected.

Pre-Course Verification Checklist

Tool	Required Action	Verification Test
Claude AI	Account created at claude.ai	Test prompt returns a response
ChatGPT	Account created at chatgpt.com	Test prompt returns a response
Cursor IDE	Downloaded and installed	Editor interface visible
Kaggle	Account created with phone verification	kaggle.com accessible with notebooks enabled
GitHub	Account created	Four repositories bookmarked
Python 3.12+	Installed with PATH (optional)	python --version displays 3.12+

All items verified? Proceed to Day 1.

Day 1: Introduction to AI Agents

GOOGLEADSAGENT.AI

googleadsagent.ai is a production AI agent—not a chatbot, dashboard, or browser extension. It has a brain (Claude by Anthropic), tools (28 custom Python API actions with live Google Ads API access), and an orchestration loop (6 Disney-named sub-agents: 🦁 Simba for reporting, 🐠 Nemo for research, ❄️ Elsa for optimization, 🧞 Aladdin for Shopping/PMAX, 🌊 Moana for creative, and 🚑 Baymax for creative innovation). Today covers the theory underlying every component.

Defining AI Agents

An AI agent is software that uses a large language model (LLM) as its reasoning engine to autonomously perceive its environment, make decisions, and take actions to accomplish goals (Google, 2025). The critical distinction from a chatbot is autonomy: agents decompose goals into subtasks, select appropriate tools, execute operations, evaluate results, and iterate until the objective is achieved.

CITATION

“This combination of reasoning, logic, and access to external information that are all connected to a Generative AI model invokes the concept of an agent, or a program that extends beyond the standalone capabilities of a Generative AI model.”

— Google, “Introduction to Agents” White Paper (Kaggle, 2025) <https://kaggle.com/whitepaper-agents>

Consider the following analogy. When you ask a chatbot about vacation destinations, it describes beaches. When you give an agent the same request, it researches flights, compares prices against your calendar, books the optimal option, reserves a hotel, adds calendar entries, and sends confirmation—all autonomously. The chatbot generates text. The agent generates outcomes.

The Three Components of Agent Architecture

Component 1: The Brain (Foundation Model)

The foundation model—Claude, GPT-4, or Gemini—serves as the reasoning engine. It interprets user intent, formulates plans, selects tools, and determines when an objective is complete. In googleadsagent.ai, Claude (Opus 4.5 for complex reasoning, Sonnet 4.5 for creative tasks) processes user goals such as “audit my Google Ads account” and orchestrates the appropriate sub-agents.

Component 2: The Tools

Tools are executable functions that enable the agent to interact with external systems: APIs, databases, search engines, file systems, and communication platforms. Without tools, the model can reason but cannot act. googleadsagent.ai provides 28 custom Python API actions—each a standalone file in the actions/main-agent/ directory—covering campaign management, keyword analysis, budget allocation, audience targeting, conversion tracking, creative management, and more.

Category	Example Actions	Count
Campaign Management	Budget Manager, RSA Ad Manager, Campaign Creator	6
Analysis	Search Term Manager, Device Performance, Change History	5

Optimization	Recommendations Manager, Bidding Strategy, Ad Schedule	4
Organization	Label Manager, Negative Keywords, Asset Manager	3
Targeting	Audience Manager, Geo & Location Manager	2
Infrastructure	API Gateway, Package Installer, Session Manager	6
Creative	Cloudinary Creative Tools	1
PMax	PMax Asset Group Manager	1

Component 3: The Loop (Orchestration / ReAct)

The orchestration layer maintains the agent's operational cycle: Reason → Act → Observe → Repeat. This pattern, called the ReAct loop (Yao et al., 2022), enables the agent to pursue multi-step objectives without human intervention at each stage. googleadsagent.ai implements this through its “The Loop” architecture:

1. User submits a question or objective.
2. CEP Protocol: Agent asks clarifying questions before any API call (Which account? Date range? Spend threshold?).
3. Top-Down Reporting: Account summary first, then drill-down through 6-tier filter system.
4. Adapter Layer: load action → inject secrets → filter params → execute via tool_executor.py.
5. Google Ads API: Real data, real mutations, all costs in dollars.
6. Claude analyzes results, validates completeness against account totals.
7. Write Safety: Writes require CONFIRM with preview of current vs. proposed + rollback labeling.
8. Heavy tasks delegated to the appropriate Disney-named sub-agent.

Chatbot vs. Agent: Definitive Comparison

Dimension	Chatbot	AI Agent
Interaction Model	Reactive (responds to each prompt)	Goal-directed (pursues objectives autonomously)
Memory	Single conversation only	Persistent across sessions
Tool Access	None (generates text only)	APIs, databases, search, file systems
Error Recovery	States “I don’t know”	Attempts alternative approaches, adapts strategy
Output Type	Text responses	Real-world actions (bookings, data changes, reports)

Verification	None	Cross-validates outputs against source data
Example	Claude answering a question	googleadsagent.ai auditing a \$50K/month account

Agent Architecture Patterns

Pattern	Mechanism	Optimal Use Case	Implementation Example
ReAct (Single Agent)	One LLM in a Think→Act→Observe loop	Focused, well-defined tasks	Simple search or calculation agent
Router	One LLM classifies and delegates to specialists	Multi-domain request handling	Customer service triage
Parallel Multi-Agent	Multiple agents execute concurrently	Independent, time-sensitive tasks	Simultaneous research from multiple sources
Sequential Multi-Agent	Pipeline: output of Agent A becomes input for Agent B	Complex dependent workflows	googleadsagent.ai's sub-agent chain
Hierarchical	Manager agent delegates to worker agents	Large-scale enterprise operations	Fully automated campaign management

Required Reading

Primary Source: Google & Kaggle. (2025). "Introduction to Agents." White Paper, 42 pages. kaggle.com/whitepaper-agents

Supplementary: Microsoft. (2025). "AI Agents for Beginners." Lessons 1-3. [GitHub](#)

Day 1 Resource Matrix

Resource	Format	Cost	URL
Google/Kaggle: Introduction to Agents	White Paper (42 pp.)	Free	kaggle.com/whitepaper-agents
Microsoft: AI Agents for Beginners	Course (12 lessons)	Free	github.com/microsoft/ai-agents-for-beginners
Hugging Face: AI Agents Course	Course + Certificate	Free	huggingface.co/learn/agents-course
DeepLearning.AI: Agents in LangGraph	Short Course	Free	deeplearning.ai/short-courses
Agent Academy	Course + Certificate	Free	agentacademy.ai
Udemy: AI Agents Crash Course (Kane)	Video Course	~\$15	udemy.com

Hands-On Lab 1A: Agent-Style Reasoning

This lab demonstrates how structured prompting elicits agent-like behavior from a standard chat interface.

1. Open Claude at claude.ai. Start a new conversation.
2. Enter the following prompt:

```
You are a Google Ads Audit Agent. A client runs 15 Search campaigns at $50,000/month. CPA has increased 40% over 3 months.

Create a systematic audit plan. For each step, specify:
(a) The exact data required
(b) The API endpoint or tool that retrieves it
(c) The specific metrics and red flags to evaluate
(d) Conditional recommendations based on findings

Structure your response as a numbered action plan.
```

WHAT YOU SHOULD SEE

Claude produces a 6–10 step audit plan. Each step specifies data requirements, API endpoints (`get_campaign_data`, `analyze_search_terms`, etc.), metrics to evaluate (Quality Score, impression share, search term match rates), and conditional recommendations. This mirrors googleadsagent.ai's Audit Agent workflow—the difference being that googleadsagent.ai executes these steps against live API data rather than reasoning about them hypothetically.

Vibe Coding Project: Tetris

This project demonstrates vibe coding—constructing functional software through natural-language description. No programming knowledge is required.

Build in Claude (Recommended)

1. Open Claude. Start a new conversation.
2. Enter the following prompt:

```
Build a fully playable Tetris game as a single HTML file with inline CSS and JavaScript. Requirements:
- Classic Tetris rules: 7 tetromino pieces, rotation, line clearing
- Arrow keys to move/rotate, spacebar for hard drop
- Score display, next piece preview, level progression
- Game over detection with restart capability
- Dark background with colorful pieces and smooth animations
- Ghost piece indicating landing position
Optimize for a polished arcade experience.
```

3. Claude generates the code and renders a live preview in the Artifact panel. The game is immediately playable.
4. To save: select the three-dot menu on the artifact, then “Copy” or download. Save as `tetris.html`. Open in any browser.
5. To iterate: provide follow-up instructions such as “Add hold-piece functionality (C key)” or “Implement touch controls for mobile.”

WHAT YOU SHOULD SEE

A fully playable Tetris game renders in Claude's right panel. Dark board (10×20 grid) with colorful tetrominoes falling. Arrow keys control movement and rotation; spacebar triggers hard drop. Score counter increments. Next-piece preview displays. Completed lines flash and clear with animation. Ghost piece shows the landing position. Game Over screen appears when the stack reaches the top, with a Restart button.

Alternative: Build in Cursor

1. Open Cursor. Create a project folder named "tetris-game."
2. Open the AI Chat panel (Cmd+I / Ctrl+I).
3. Paste the identical Tetris prompt.
4. The AI generates index.html in the File Explorer. Right-click → Reveal in Finder/Explorer → double-click to play in browser.

Day 1 Self-Assessment

1. Explain the ReAct loop to a non-technical colleague using an analogy.
2. Identify three functional differences between a chatbot and an AI agent.
3. Map the Brain, Tools, and Loop to specific googleadsagent.ai components.
4. Describe a scenario where an agent is unnecessarily complex and a direct prompt suffices.
5. Name the 6 sub-agents in googleadsagent.ai and state each one's specialization.

Day 2: Agent Tools & Interoperability With MCP



GOOGLEADSAGENT.AI

Each of googleadsagent.ai's 28 API actions is a tool. The adapter layer (tool_executor.py) loads actions, injects credentials, filters parameters, and executes—providing a standardized interface between the brain and the Google Ads API. MCP formalizes this pattern as an industry standard. Today covers tool architecture, the tool-use cycle, and MCP's role in making agent tools universally interoperable.

Understanding Agent Tools

A tool, in the context of AI agents, is any executable function that extends the model's capabilities beyond text generation. Tools enable agents to retrieve real-time data, modify external systems, perform calculations, and interact with services. The distinction between a capable chatbot and a functional agent is the presence and effective use of tools.

The Tool-Use Execution Cycle

When an agent invokes a tool, six discrete steps occur:

- 1. Goal Reception:** The user provides an objective ("Identify wasted spend in my Google Ads account").
- 2. Reasoning:** The LLM determines which tool is required ("I need to call the Search Term Manager to pull search query data").
- 3. Structured Tool Call:** The LLM outputs a formatted request: { action: "search_term_manager", params: { date_range: "30d", campaign_ids: ["123"] } }.
- 4. Execution:** The application infrastructure (not the LLM) processes the request. In googleadsagent.ai, tool_executor.py loads the action file, injects API credentials, and calls the Google Ads API.
- 5. Result Injection:** Raw API data (search terms, costs, conversion counts) is returned to the LLM's context window.
- 6. Synthesis:** The LLM interprets the data and produces a human-readable analysis: "You're spending \$3,200/month on irrelevant search terms. Here are the top 15 negative keyword recommendations."

Tool Taxonomy in googleadsagent.ai

Category	Actions	Tool Names	Count
Campaign	Create, manage, modify campaigns and ad groups	Budget Manager, Campaign Creator, RSA Ad Manager	6
Analysis	Pull and analyze performance data	Search Term Manager, Query Planner, Device Performance	5
Optimization	Improve account performance	Recommendations Manager, Bidding	4

		Strategy, Experiments	
Organization	Structure and label account elements	Label Manager, Negative Keywords, Asset Manager	3
Targeting	Define and refine audiences	Audience Manager, Geo & Location Manager	2
Infrastructure	System operations and connectivity	API Gateway, Session Manager, Package Installer	6
Creative	Visual asset management	Cloudinary Creative Tools	1
Performance Max	Shopping and PMax campaigns	PMAX Asset Group Manager	1

Model Context Protocol (MCP)

MCP, the Model Context Protocol, is an open standard created by Anthropic in November 2024 that provides a universal interface for connecting AI models to external tools and data sources. The analogy is USB-C: before USB-C, every device required a unique cable. Before MCP, every AI application required custom integration code for every data source. MCP standardizes the connection.

MCP has been adopted by Google (“Our Gemini API and SDK are now compatible with MCP tools,” Pichai, 2025), OpenAI, Microsoft, and the broader AI development community.

CITATION

“We’re excited to announce that our Gemini API and SDK are now compatible with MCP tools.”
— Sundar Pichai, Google I/O 2025 <https://blog.google/innovation-and-ai/technology/ai/io-2025-keynote/>

MCP Architecture

MCP Host: The AI application (Claude Desktop, Cursor, a custom application). This is the user-facing interface.

MCP Client: Embedded within the host. Manages connections to one or more MCP servers.

MCP Server: A lightweight service exposing three capability types: Tools (executable functions), Resources (readable data), and Prompts (task templates).

PRO TIP

Consider the implications for googleadsagent.ai: if its 28 API actions were packaged as an MCP server, any MCP-compatible AI application—Claude, ChatGPT, Cursor, or any future tool—could instantly manage Google Ads accounts without custom integration code.

Day 2 Resource Matrix

Resource	Format	Cost	URL
Google/Kaggle: Agent Tools & MCP	White Paper	Free	Kaggle Agents Intensive

MCP Official Documentation	Technical Docs	Free	modelcontextprotocol.io
MCP GitHub SDKs	Open Source	Free	github.com/modelcontextprotocol
Anthropic MCP Announcement	Blog Post	Free	anthropic.com/news/model-context-protocol
MCP Server Directory	Directory	Free	mcp.so
Microsoft Lessons 4–6	Course	Free	github.com/microsoft/ai-agents-for-beginners

Hands-On Lab 2A: Observing Tool Use

1. Open Claude. Start a new conversation.
2. Enter the following prompt:

```
Search the web for the latest MCP (Model Context Protocol) announcements from the past 30 days. Summarize the top 3 developments and analyze implications for digital advertising.
```

WHAT YOU SHOULD SEE

Claude activates its web search tool—a “Searching...” indicator appears. Results return with source citations. Claude formulated a search query, executed the tool, received results, and synthesized them into analysis. This is the identical pattern powering every one of googleadsagent.ai’s 28 API actions.

Hands-On Lab 2B: Designing a Tool Schema

```
Design a tool schema for a Google Ads Search Term Analyzer. Specify: (1) tool name and description, (2) typed input parameters, (3) structured output format, (4) error cases. Format as JSON suitable for an MCP server definition.
```

WHAT YOU SHOULD SEE

Claude produces a complete JSON schema with tool name, description, typed parameters (date_range as string, campaign_ids as array, cost_threshold as number), structured output (arrays of search terms with cost, clicks, conversions, match_type), and documented error cases (invalid_campaign_id, api_rate_limit, insufficient_permissions).

Vibe Coding Project: Zelda-Style Adventure

1. Open Claude. New conversation.
2. Enter:

```
Build a Zelda-style top-down adventure game as a single HTML file. Requirements:
- Player character controlled by arrow keys or WASD
- Overworld map: grass, collidable trees, impassable water, paths
- Three or more enemy types with movement patterns and damage
- Sword attack on spacebar with visible swing animation
- Health system (3 hearts), collectible rupees and health pickups
- Multiple connected rooms with edge-of-screen transitions
- Mini-map overlay showing explored rooms
- SNES-era pixel-art aesthetic with green overworld palette
```

 WHAT YOU SHOULD SEE

A top-down adventure game with a green overworld. Character sprite on grass with trees and rocks. Enemies patrol in patterns. Hearts display in the top-left. Rupee counter shows score. Spacebar triggers a visible sword arc. Walking to the screen edge transitions to an adjacent room. Mini-map in the corner shows explored areas.

Day 2 Self-Assessment

1. Explain MCP to a non-technical stakeholder using the USB-C analogy.
2. Name the three MCP architectural components and their roles.
3. Trace how googleadsagent.ai's Search Term Manager follows the tool-use cycle.
4. Explain the advantage of MCP over per-application custom API integrations.
5. Identify two MCP servers that would benefit a paid media professional.

Day 3: Context Engineering — Sessions & Memory

GOOGLEADSAGENT.AI

googleadsagent.ai's CEP Protocol (Clarifying Exchange Protocol) is context engineering in action: before any API call, the agent asks clarifying questions to build the right context (Which account? ENABLED only? Date range? Spend threshold?). The Session & State Manager (Action #17) maintains state across multi-step analyses. Each sub-agent writes findings to shared context that the orchestrator synthesizes.

Defining Context Engineering

Context engineering is the discipline of designing and managing the complete information payload that flows into an AI agent's prompt at each step of execution. It determines what the agent knows, what it remembers, and how it personalizes its behavior (Google, 2025).

CITATION

"Context Engineering is concerned with the entire payload, dynamically constructing a state-aware prompt based on the user, conversation, and external data... transforming an agent from being an expert on facts to an expert on the user."

— Google, "Context Engineering: Sessions & Memory" White Paper <https://kaggle.com/whitepaper-context-engineering-sessions-and-memory>

The Six Layers of Agent Context

Layer	Contents	googleadsagent.ai Implementation
1. System Instructions	Agent identity, behavioral rules, capabilities	Main agent system prompt defining The Loop methodology
2. User Profile	User identity, preferences, access level	Account ID, credential pattern (A/B/C/D), user permissions
3. Short-Term Memory	Current conversation messages	Active conversation thread with clarifying Q&A
4. Long-Term Memory	Persistent facts across sessions	Previous audit findings, account historical baselines
5. Retrieved Context (RAG)	Relevant documents pulled on demand	Google Ads documentation, best practices database
6. Tool Results	Data returned from tool execution	Live campaign metrics, search term reports, budget data

The Scratchpad Pattern

The scratchpad pattern is among the most effective strategies for managing multi-step agent workflows. Rather than relying on raw conversation history (which can be noisy and exceed context limits), the agent maintains a structured working document with categorized findings. Each processing step appends to the scratchpad rather than re-processing the entire thread.

In googleadsagent.ai, the orchestrator maintains sections corresponding to each sub-agent's domain. When 🦁 Simba (Reporting) completes its analysis, it writes findings to the Reporting section. When ❄️

Elsa (Optimization) identifies opportunities, they go to the Optimization section. The final synthesis reads the complete scratchpad rather than reprocessing every intermediate conversation turn.

RAG: Retrieval-Augmented Generation

RAG enables agents to access knowledge bases that exceed the context window's capacity. Instead of pre-loading every document into the prompt, the agent queries a vector database and retrieves only semantically relevant chunks. This grounds responses in authoritative source material rather than the model's potentially outdated training data.

Memory Management Strategies

Strategy	Mechanism	Advantage	Limitation
Full History	Every message included in each call	Maximum accuracy	Context window overflow on long sessions
Sliding Window	Retain only the most recent N messages	Bounded memory usage	Loses early context
Progressive Summarization	Condense older messages into summaries	Preserves key facts efficiently	Summarization introduces information loss
Scratchpad	Structured working document of key findings	Best efficiency-accuracy balance	Requires careful schema design
RAG	Vector database retrieval of relevant chunks	Scales to massive knowledge bases	Requires infrastructure setup

Day 3 Resource Matrix

Resource	Format	Cost	URL
Google/Kaggle: Context Engineering	White Paper	Free	Kaggle
LangChain Memory Documentation	Technical Docs	Free	langchain.com
LlamaIndex RAG Tutorial	Tutorial	Free	llamaindex.ai
Simon Willison: Context Engineering	Blog Post	Free	simonwillison.net
Microsoft Lessons 7-9	Course	Free	github.com/microsoft/ai-agents-for-beginners
Pinecone Vector DB Guide	Tutorial	Free	pinecone.io

Vibe Coding Project: Football (Tecmo Bowl)

1. Open Claude. New conversation.
2. Enter:

```
Build an American football arcade game (Tecmo Bowl style) as a
single HTML file. Requirements:
- Top-down football field with yard lines and end zones
- Player controls quarterback/ball carrier with arrow keys
- 5 offensive and 5 defensive players per play
- Play selection screen with 4 offensive plays
- Pass mechanic: spacebar to throw, arrow keys to aim
- Tackle detection: defender sprite contact with ball carrier
- Complete scoring, downs system (1st-4th), and yard tracking
- Touchdown celebration animation
- 8-bit pixel art aesthetic
```

WHAT YOU SHOULD SEE

Top-down football field with green turf, white yard lines, colored end zones. Play selection screen offers four options. After selection, players line up in formation. Arrow keys control movement; spacebar throws. Defenders pursue. Score, quarter, down-and-distance display at top.

Day 3 Self-Assessment

1. Distinguish session memory from long-term memory with examples.
2. Describe the scratchpad pattern and its implementation in googleadsagent.ai.
3. List the six context layers and provide a concrete example for each.
4. Define RAG and explain why agents require it.
5. Explain googleadsagent.ai's CEP Protocol and its role in context engineering.

Day 4: Agent Quality — Evaluation & Observability

GOOGLEADSAGENT.AI

googleadsagent.ai implements quality through multiple mechanisms: (1) Top-Down Reporting validates that detailed metrics sum to account totals, preventing hallucinated numbers; (2) the Write Safety protocol requires CONFIRM with a preview of current vs. proposed changes plus rollback labeling before any mutation; (3) each sub-agent has a governing principle (🦁 Simba: “Summarize, don’t dump”; ❄️ Elsa: “Preview before execute”).

The Agent Quality Problem

The fundamental challenge with AI agents in production is reliability. A model that hallucinates 5% of the time in a casual conversation is a minor inconvenience. A model that hallucinates 5% of the time while managing \$50,000/month in advertising spend is a financial liability. Agent quality engineering addresses this gap through systematic evaluation, continuous monitoring, and proactive safety constraints.

Three Pillars of Agent Quality

Pillar 1: Evaluation

Unit Evaluation: Test individual tool calls in isolation. Does the Search Term Manager return accurate data? Does the Budget Manager calculate reallocations correctly?

Trajectory Evaluation: Test the agent’s reasoning path. Did it select appropriate tools? Did it execute them in a logical sequence? Did it stop when sufficient data was gathered?

Final Response Evaluation: Test the delivered output. Is the audit report accurate? Are recommendations actionable and supported by the underlying data?

Pillar 2: Observability

Logging: Timestamped records of every tool call, LLM inference, and decision point. Analogous to an aircraft flight recorder.

Tracing: End-to-end path through a request, from user input through all intermediate tool calls and reasoning steps to final output.

Metrics: Quantitative indicators: latency, token consumption, tool call success rate, user satisfaction, factual accuracy.

Pillar 3: Guardrails

Input Guardrails: Validate that user requests fall within the agent’s authorized scope before processing begins.

Output Guardrails: Verify agent responses for accuracy and appropriateness before delivery or execution.

Action Guardrails: Require explicit human approval for high-impact operations. In googleadsagent.ai, all write operations (bid changes, campaign pausing, keyword additions) trigger the CONFIRM protocol: the

agent displays current state vs. proposed change, explains rationale, and waits for user confirmation before executing.

LLM-as-a-Judge Evaluation

LLM-as-a-Judge is an evaluation methodology where a separate language model scores agent outputs against defined rubrics. The evaluating model receives the original query, the agent's response, and scoring criteria (accuracy, completeness, relevance, safety), then returns structured scores with justification. Research demonstrates that LLM judges achieve high correlation with human evaluators while operating at dramatically lower cost and higher throughput.

Day 4 Resource Matrix

Resource	Format	Cost	URL
Google/Kaggle: Agent Quality	White Paper	Free	Kaggle
LangSmith	Observability Platform	Free tier	langsmith.com
Braintrust	Evaluation Platform	Free tier	braintrust.dev
Arize Phoenix	Open Source Tracing	Free	github.com/Arize-ai/phoenix
RAGAS	RAG Evaluation Framework	Free	ragas.io
Microsoft Lessons 10–11	Course	Free	github.com/microsoft/ai-agents-for-beginners

Vibe Coding Project: Super Mario Bros

1. Open Claude. New conversation.
2. Enter:

```
Build a Super Mario Bros-style platformer as a single HTML file.
- Side-scrolling with controllable character
- Arrow keys for movement, spacebar or up-arrow to jump
- Platforms, pipes, brick blocks, question-mark blocks
- Breakable blocks yield coins when hit from below
- Two or more enemy types; jump on enemies to defeat them
- Coin counter, score display, pit death, 3-life system
- Flag pole to complete the level
- Authentic NES-era physics: momentum, gravity, acceleration
```

WHAT YOU SHOULD SEE

A side-scrolling platformer with blue sky, green pipes, brown bricks, and question-mark blocks. Arrow keys run (camera scrolls), spacebar jumps. Hit blocks from below for coins. Jump on enemies to defeat. Fall in a pit—lose a life. Reach the flagpole to clear the level. Score and coin counter at top. Lives in the corner.

Day 4 Self-Assessment

1. Name the three pillars of agent quality and provide an example of each.

2. Explain the LLM-as-a-Judge methodology in one paragraph.
3. Why is hallucination particularly dangerous for agents managing advertising budgets?
4. Describe googleadsagent.ai's Write Safety / CONFIRM protocol.
5. Name two observability platforms and their primary function.

Day 5: Prototype to Production & Multi-Agent Systems

GOOGLEADSAGENT.AI

googleadsagent.ai is a multi-agent system with 6 Disney-named sub-agents, each running specialized models: 🦁 Simba (Opus 4.5, 8 actions): Reporting & Analysis — “Summarize, don’t dump.” 🐠 Nemo (Opus 4.5, 4 actions): Research & Intelligence — “Insight over information.” ❄️ Elsa (Opus 4.5, 8 actions): Optimization — “Preview before execute.” 🧞 Aladdin (Opus 4.5, 7 actions): Shopping & PMax — “ROAS is king.” 🌊 Moana (Opus 4.5, 2 actions): Creative — “Visual first.” 🏥 Baymax (Sonnet 4.5, 2 actions): Creative Innovation — “Receive → Process → Return.”

The Prototype-to-Production Gap

Building an agent that functions in a demonstration is analogous to cooking a meal for friends. Deploying an agent in production is analogous to opening a restaurant. The output may be identical, but the infrastructure requirements differ by orders of magnitude.

Dimension	Prototype	Production
Error Handling	Crash and debug manually	Graceful fallbacks, retry logic, automated alerting
Authentication	Hardcoded API keys	OAuth 2.0, key rotation, least-privilege credential patterns
Monitoring	Manual log inspection	Real-time dashboards, anomaly detection, SLA tracking
Scaling	Single concurrent user	Queue management, rate limiting, horizontal scaling
Testing	Manual verification	Automated test suites, regression testing, CI/CD pipelines
Data Privacy	Test/synthetic data	PII handling, encryption at rest/transit, compliance (GDPR, CCPA)
Cost Management	Unbounded spending	Token budgets, model routing, cost alerts
Human Oversight	None	Approval workflows for critical actions (CONFIRM protocol)

Agent-to-Agent Protocol (A2A)

A2A, the Agent-to-Agent Protocol, is an open standard created by Google in April 2025 that enables AI agents to communicate and collaborate regardless of their underlying framework, vendor, or programming language. If MCP is the USB-C for connecting agents to tools, A2A is the telephone network for connecting agents to each other.

CITATION

“A2A focuses on enabling agents to collaborate in their natural, unstructured modalities, even when they don’t share memory, tools and context. We are enabling true multi-agent scenarios without limiting an agent to a ‘tool.’”

— Google Developers Blog, April 2025 <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>

MCP vs. A2A: Complementary Protocols

Dimension	MCP (Model Context Protocol)	A2A (Agent-to-Agent Protocol)
Purpose	Connect AI models to tools and data	Connect AI agents to each other
Analogy	USB-C (peripheral connectivity)	Telephone network (entity-to-entity communication)
Relationship Model	Client → Server (hierarchical)	Peer → Peer (collaborative)
Creator	Anthropic (November 2024)	Google (April 2025)
In googleadsagent.ai	28 tool actions the agents invoke	Sub-agents collaborating on complex audits

CITATION

“Agent2Agent (A2A) is an open standard protocol that gives your agents a common language to collaborate no matter what framework or vendor they are built on.”

— Google Cloud Next 2025 <https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/next-2025/>

googleadsagent.ai: Architecture Walkthrough

The following traces how googleadsagent.ai processes a real request: “Audit my Google Ads account and identify opportunities to reduce wasted spend.”

- 1. User → Orchestrator:** The main agent (Claude Opus 4.5) receives the request and activates the CEP Protocol: “Which account? ENABLED campaigns only? What date range? Minimum spend threshold?”
- 2. Orchestrator →  Simba (Reporting):** Simba pulls account-level performance data using the Query Planner and Device Performance actions. Principle: “Summarize, don’t dump.” Writes to scratchpad: “3 campaigns with zero conversions in 30 days, \$2,400 total spend.”
- 3. Orchestrator →  Nemo (Research):** Nemo analyzes search terms and competitive landscape using Search Term Manager and web research. Principle: “Insight over information.” Finds: “62% irrelevant search terms. ‘free’ appears in 28% of triggering queries.”
- 4. Orchestrator →  Elsa (Optimization):** Elsa evaluates bidding strategies, budget allocation, and recommendations. Principle: “Preview before execute.” Identifies: “Reallocating \$3,200/month from underperformers to top campaigns could increase ROAS by 28%.”
- 5. Orchestrator →  Moana (Creative):** Moana reviews ad creative assets using the RSA Ad Manager and Cloudinary tools. Principle: “Visual first.” Reports: “8 ads with ‘Poor’ or ‘Average’ strength. 4 campaigns missing responsive search ads.”
- 6. Orchestrator Synthesizes:** The main agent reads the complete scratchpad, cross-references findings across sub-agents, validates that detailed figures sum to account totals (Top-Down

Reporting), resolves contradictions, and compiles a unified audit report with prioritized recommendations, supporting data, and estimated financial impact.

Google's Agentic Commerce Vision

CITATION

Announced the Universal Commerce Protocol (UCP) for agentic commerce, compatible with A2A, Agent Payments Protocol, and MCP. Built with Shopify, Etsy, Wayfair, Target, and Walmart.

— Sundar Pichai, NRF 2026 (January 12, 2026)

<https://blog.google/company-news/inside-google/message-ceo/nrf-2026-remarks/>

The strategic trajectory is clear: Google is constructing infrastructure for AI agents to participate directly in commerce—discovering products, comparing prices, negotiating, and purchasing on behalf of consumers. For advertising professionals, this implies that future advertisements may be evaluated and acted upon by AI agents rather than human consumers. Practitioners who understand agent architecture today will be positioned to adapt.

Day 5 Resource Matrix

Resource	Format	Cost	URL
Google/Kaggle: Prototype to Production	White Paper	Free	Kaggle
A2A Protocol Documentation	Specification	Free	google.github.io/A2A
Google Agent Development Kit (ADK)	Open Source SDK	Free	github.com/google/adk-python
CrewAI Framework	Open Source	Free	crewai.com
LangGraph Multi-Agent	Framework	Free	langchain.com/langgraph
googleadsagent.ai	Live Agent	Free	googleadsagent.ai

Vibe Coding Project: Duck Hunt

1. Open Claude. New conversation.
2. Enter:

```
Build a Duck Hunt-style shooting gallery as a single HTML file.
- Click or tap to shoot at flying ducks
- Ducks enter from sides with randomized flight patterns
- Crosshair cursor replaces default mouse pointer
- Gunshot sound and muzzle flash via Web Audio API
- Hit ducks tumble with falling animation
- Hunting dog at bottom retrieves fallen ducks
- Round system: 10 ducks per round, 3 rounds, increasing difficulty
- Ammo limit: 3 shots per duck
- Score display and accuracy percentage
- Classic NES aesthetic
```

WHAT YOU SHOULD SEE

Shooting gallery with blue sky and grass. Pixelated dog in grass. Ducks fly in arcing patterns. Crosshair replaces cursor. Click produces gunshot sound and flash. Hit ducks tumble downward; dog jumps to catch. Score and ammo at top. Miss all shots and the dog laughs. Rounds progress with faster, more erratic flight patterns.

Capstone Project: AI-Powered Game Master RPG

The capstone project integrates every concept from the course into a single application: an interactive text RPG where an AI Game Master reasons, manages state, executes game mechanics, and adapts to player decisions—a complete agentic system.

```
Build an AI-powered text RPG Game Master as a single HTML file.  
- Character creation: name, class (Warrior/Mage/Rogue)  
- Persistent stats: HP, Attack, Defense, Magic, Level, XP  
- Inventory system with equippable items and consumables  
- Turn-based combat with enemy AI and attack patterns  
- Narrative that adapts to player choices (3+ branching points)  
- Shop between encounters for purchasing equipment  
- Boss encounter requiring strategic decision-making  
- Visual HP bars, atmospheric dark-fantasy styling  
- Save/load game state
```

WHAT YOU SHOULD SEE

Dark fantasy RPG interface. Character card with name, class, and stat bars (HP red, XP gold, Mana blue). Center panel displays narrative text in atmospheric font. Below: 2–4 choice buttons for player decisions. Right panel shows inventory with item icons. During combat, enemy stats appear with HP bars and action log. A complete agentic system: reasoning (game AI), tools (combat calculator, inventory manager, story engine), memory (character state and story progress), and orchestration (managing all subsystems simultaneously).

Day 5 Self-Assessment

1. Contrast MCP and A2A using the USB-C vs. telephone-network analogy.
2. Trace how googleadsagent.ai's 6 sub-agents collaborate on a comprehensive audit.
3. Identify five differences between prototype and production agent deployments.
4. Explain Google's Universal Commerce Protocol and its implications for advertisers.
5. Name each Disney sub-agent and state its governing principle.

Appendix A: Complete Resource Library

Google/Kaggle White Papers (Free)

#	Title	Pages	Primary Access	Alternative
1	Introduction to Agents	42	kaggle.com/whitepaper-agents	GitHub compilation
2	Agent Tools & Interoperability with MCP	—	Google Agents Intensive	GitHub compilation
3	Context Engineering: Sessions & Memory	—	Kaggle direct link	GitHub compilation
4	Agent Quality	—	Google Agents Intensive	GitHub compilation
5	Prototype to Production	—	Google Agents Intensive	GitHub compilation

Complete compilation: github.com/sameerjadhav/google-agents-resources

Free Courses and Certifications

Provider	Course Title	Format	Certificate	URL
Microsoft	AI Agents for Beginners (12 Lessons)	Video + Code	No	github.com/microsoft/ai-agents-for-beginners
Hugging Face	AI Agents Course	Video + Code	Yes (Free)	huggingface.co/learn/agents-course
DeepLearning.AI	AI Agents in LangGraph	Short Course	Yes (Free)	deeplearning.ai/short-courses
Agent Academy	Understanding Agentic AI	Video + Code	Yes (Free)	agentacademy.ai
Google	Kaggle Agents Intensive	Notebooks	Yes	kaggle.com

Protocols and Specifications

Protocol	Creator	Date	Documentation	GitHub
MCP (Model Context Protocol)	Anthropic	November 2024	modelcontextprotocol.io	github.com/modelcontextprotocol
A2A (Agent-to-Agent)	Google	April 2025	google.github.io/A2A	github.com/a2aproject/A2A

UCP (Universal Commerce Protocol)	Google	January 2026	Announced at NRF 2026	—
-----------------------------------	--------	--------------	-----------------------	---

Agent Development Frameworks

Framework	Focus	Language	License	URL
Google ADK	Agent Development Kit	Python	Apache 2.0	github.com/google/adk-python
LangGraph	Multi-agent orchestration	Python/JS	MIT	langchain.com/langgraph
CrewAI	Agent team orchestration	Python	MIT	crewai.com
AutoGen	Multi-agent conversation	Python	MIT	github.com/microsoft/autogen
Semantic Kernel	Enterprise agent SDK	C#/Python/Java	MIT	github.com/microsoft/semantic-kernel

Observability and Evaluation Platforms

Platform	Function	Pricing	URL
LangSmith	LLM monitoring and tracing	Free tier	langsmith.com
Braintrust	LLM evaluation and evals	Free tier	braintrust.dev
Arize Phoenix	Open source tracing	Free (OSS)	github.com/Arize-ai/phoenix
RAGAS	RAG evaluation framework	Free (OSS)	ragas.io
Weights & Biases	ML experiment tracking	Free tier	wandb.ai

Vibe Coding Game Projects

Day	Game	Genre	Primary Tool	Complexity
1	Tetris	Arcade Puzzle	Claude Artifacts	Introductory
2	Zelda-Style Adventure	Top-Down RPG	Claude / Cursor	Intermediate
3	Football (Tecmo Bowl)	Sports Arcade	Claude / Cursor	Intermediate
4	Super Mario Bros	Side-Scrolling	Claude / Cursor	Advanced

Platformer				
5	Duck Hunt	Shooting Gallery	Claude / Cursor	Intermediate
Capstone	AI Game Master RPG	Text RPG + Agent	Claude / Cursor	Advanced

Official Citations Index

Source	Event/Publication	Date	URL
Sundar Pichai	Google I/O 2025 Keynote	May 20, 2025	blog.google/io-2025-keynote
Demis Hassabis	Gemini 2.0 Launch Announcement	March 6, 2025	blog.google/gemini-2
Google	Gemini 3 Launch	November 18, 2025	blog.google/gemini-3
Google	Agent2Agent Protocol Announcement	April 9, 2025	developers.googleblog.com
Google	Cloud Next 2025 Announcements	April 9, 2025	blog.google/cloud-next-2025
Sundar Pichai	NRF 2026 Remarks	January 12, 2026	blog.google/nrf-2026
Alphabet Inc.	Q2 2025 Earnings Call	July 23, 2025	abc.xyz/investor
Anthropic	MCP Announcement	November 2024	anthropic.com/news
Andrej Karpathy	Vibe Coding concept	February 2025	x.com/karpathy
Yao et al.	ReAct: Reasoning + Acting Paper	2022	arxiv.org

Appendix B: Glossary of Key Terms

Term	Definition
A2A (Agent-to-Agent Protocol)	Google's open standard enabling AI agents to communicate and collaborate regardless of framework or vendor.
Agent	Software that uses an LLM to autonomously perceive, reason, decide, and act to accomplish goals.
CEP Protocol	Clarifying Exchange Protocol; googleadsagent.ai's method of asking qualifying questions before API execution.
CONFIRM Protocol	Write safety mechanism requiring user approval before any data mutation, with current-vs-proposed preview.
Context Engineering	The discipline of designing and managing the information payload flowing into an agent's prompt.
Context Window	The maximum amount of text (measured in tokens) an LLM can process in a single interaction.
Foundation Model / LLM	Large Language Model; the neural network serving as an agent's reasoning engine (e.g., Claude, GPT-4, Gemini).
Guardrails	Safety mechanisms (input, output, action) that constrain agent behavior within authorized boundaries.
Hallucination	When an AI model confidently produces incorrect, fabricated, or ungrounded information.
LLM-as-a-Judge	Evaluation methodology using a separate LLM to score agent outputs against defined rubrics.
MCP (Model Context Protocol)	Anthropic's open standard for universally connecting AI models to tools, resources, and data sources.
Multi-Agent System	Architecture where multiple specialized agents collaborate on tasks exceeding individual agent capability.
Orchestrator	The coordinating agent or system that delegates tasks to sub-agents and synthesizes their outputs.
RAG (Retrieval-Augmented Generation)	Technique where agents search vector databases to retrieve relevant context before generating responses.
ReAct Loop	Agent architecture pattern: Reason about the next action → Act using a tool → Observe the result → Repeat.
Scratchpad Pattern	Agent maintains a structured working document of

	categorized findings during multi-step tasks.
Sub-Agent	A specialized agent focused on a single domain within a multi-agent system (e.g., 🦁 Simba for Reporting).
The Loop	googleadsagent.ai's core architecture: ask → clarify → execute → validate → report → loop until truth.
Tool	An executable function (API call, database query, calculation) that extends an agent's capabilities beyond text.
Top-Down Reporting	Validation pattern: account summary first, then drill-down; ensures details sum to totals.
UCP (Universal Commerce Protocol)	Google's standard for agentic commerce transactions, compatible with A2A, MCP, and Agent Payments Protocol.
Vibe Coding	Constructing software through natural-language description; AI generates the implementation (Karpathy, 2025).

Appendix C: googleadsagent.ai Technical Reference

System Overview

Component	Specification
Foundation Model	Claude by Anthropic (Opus 4.5 primary, Sonnet 4.5 for creative)
Custom Tools	28 Python API actions with live Google Ads API read/write access
Sub-Agents	6 Disney-named specialists: Simba, Nemo, Elsa, Aladdin, Moana, Baymax
Credential Patterns	A (5-key, 12 actions), B (4-key, 13 actions), C (3-key Cloudinary, 1), D (none, 2)
Safety	CONFIRM protocol with current-vs-proposed preview + rollback labeling
Context	CEP Protocol for pre-execution clarification; Session State Manager
Validation	Top-Down Reporting: detailed metrics cross-validated against account totals
Deployment	CLI, REST API (uvicorn), Docker, Python SDK
Repository	66 files across 12 directories
License	Open source
Cost	Free to use
URL	googleadsagent.ai

Sub-Agent Specifications

Agent	Name	Model	Actions	Domain	Governing Principle
	Simba	Opus 4.5	8	Reporting & Analysis	"Summarize, don't dump."
	Nemo	Opus 4.5	4	Research & Intelligence	"Insight over information."
	Elsa	Opus 4.5	8	Optimization	"Preview before execute."
	Aladdin	Opus 4.5	7	Shopping & PMax	"E-commerce focused. ROAS is king."
	Moana	Opus 4.5	2	Creative	"Visual first."

	Baymax	Sonnet 4.5	2	Creative Innovation	"Receive → Process → Return ready-to-use."
---	--------	------------	---	---------------------	--

Complete Action Directory (28 Actions)

#	Action Name	Category	Credential
01	Label Manager	Organization	A
02	Conversion Tracking	Analysis	A
03	Audience Manager	Targeting	B
04	Asset Manager	Organization	B
05	Budget Manager	Campaign	B
06	RSA Ad Manager	Campaign	B
07	Bid & Keyword Manager	Campaign	B
08	Negative Keywords	Organization	B
09	Campaign & Ad Group Manager	Campaign	B
10	Google Ads Mutate	Campaign	B
11	Account Access Checker	Infrastructure	B
12	Scripts Manager	Infrastructure	A
13	Experiments Manager	Optimization	A
14	Package Installer	Infrastructure	D
15	Check User Access	Infrastructure	B
16	API Gateway	Infrastructure	B
17	Session & State Manager	Infrastructure	D
18	Clouinary Creative Tools	Creative	C
19	Query Planner	Analysis	A
20	Recommendations Manager	Optimization	A
21	Search Term Manager	Analysis	B
22	Geo & Location Manager	Targeting	B
23	Device Performance	Analysis	A
24	Change History Manager	Analysis	A
25	Campaign Creator	Campaign	A

26	Ad Schedule Manager	Optimization	A
27	Bidding Strategy Manager	Optimization	A
28	PMax Asset Group Manager	PMax	A

References

- Alphabet Inc. (2025, July 23). Q2 2025 earnings call transcript. <https://abc.xyz/investor/>
- Anthropic. (2024, November). Introducing the Model Context Protocol. <https://anthropic.com/news/model-context-protocol>
- Google. (2025). Introduction to agents [White paper]. Kaggle. <https://kaggle.com/whitepaper-agents>
- Google. (2025). Agent tools and interoperability with MCP [White paper]. Kaggle Agents Intensive.
- Google. (2025). Context engineering: Sessions and memory [White paper]. Kaggle. <https://kaggle.com/whitepaper-context-engineering-sessions-and-memory>
- Google. (2025). Agent quality [White paper]. Kaggle Agents Intensive.
- Google. (2025). Prototype to production [White paper]. Kaggle Agents Intensive.
- Google. (2025, April 9). Announcing the Agent2Agent Protocol (A2A). Google Developers Blog. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- Google. (2025, April 9). Google Cloud Next 2025 announcements. Google Blog. <https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/next-2025/>
- Google DeepMind. (2025, March 6). Gemini 2.0: Built for the agentic era. Google Blog. <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/>
- Google DeepMind. (2025, November 18). Introducing Gemini 3. Google Blog.
- Karpathy, A. (2025, February). Vibe coding [Social media post]. X. <https://x.com/karpathy>
- Microsoft. (2025). AI agents for beginners [Online course, 12 lessons]. GitHub. <https://github.com/microsoft/ai-agents-for-beginners>
- Pichai, S. (2025, May 20). Google I/O 2025 keynote address. Google Blog. <https://blog.google/innovation-and-ai/technology/ai/io-2025-keynote/>
- Pichai, S. (2026, January 12). Remarks at NRF 2026. Google Blog. <https://blog.google/company-news/inside-google/message-ceo/nrf-2026-remarks/>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. arXiv. <https://arxiv.org/abs/2210.03629>

References (APA 7th Edition)

About the Author

About the Author

John Williams is a Senior Paid Media Specialist at Seer Interactive with 15+ years of experience managing \$48M+ in digital advertising spend across Google Ads, Meta, Microsoft, Amazon, and other platforms. Previous positions include NortonLifeLock (where he achieved 192% year-over-year paid search growth), Gen Digital, Avast, and Farmers Insurance.

He is the creator of googleadsagent.ai, an open-source AI agent for Google Ads management with 28 custom API actions and 6 specialized sub-agents, and operates “It All Started With A Idea,” his consultancy and product studio focused on AI-powered marketing tools and automation systems. John speaks at industry conferences including Hero Conf on AI applications in advertising and maintains an open-source GitHub repository with 19+ marketing automation tools.

John also serves as an assistant football coach at Casteel High School in Queen Creek, Arizona, coaching wide receivers, defensive backs, and special teams. He is a former Washington State University football player (2002–2005). He draws frequent parallels between coaching methodology and business strategy—both disciplines demand systematic preparation, real-time pattern recognition, and decisive action under uncertainty.

This course is free and open source.

Share it. Teach it. Fork it on GitHub.

The future of AI agents belongs to everyone.

googleadsagent.ai

Version 3.0 • February 2026 • Resources verified at time of publication